

Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

YOLO and other CNNs

Oct 22, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

Autonomous, Cooperating Systems

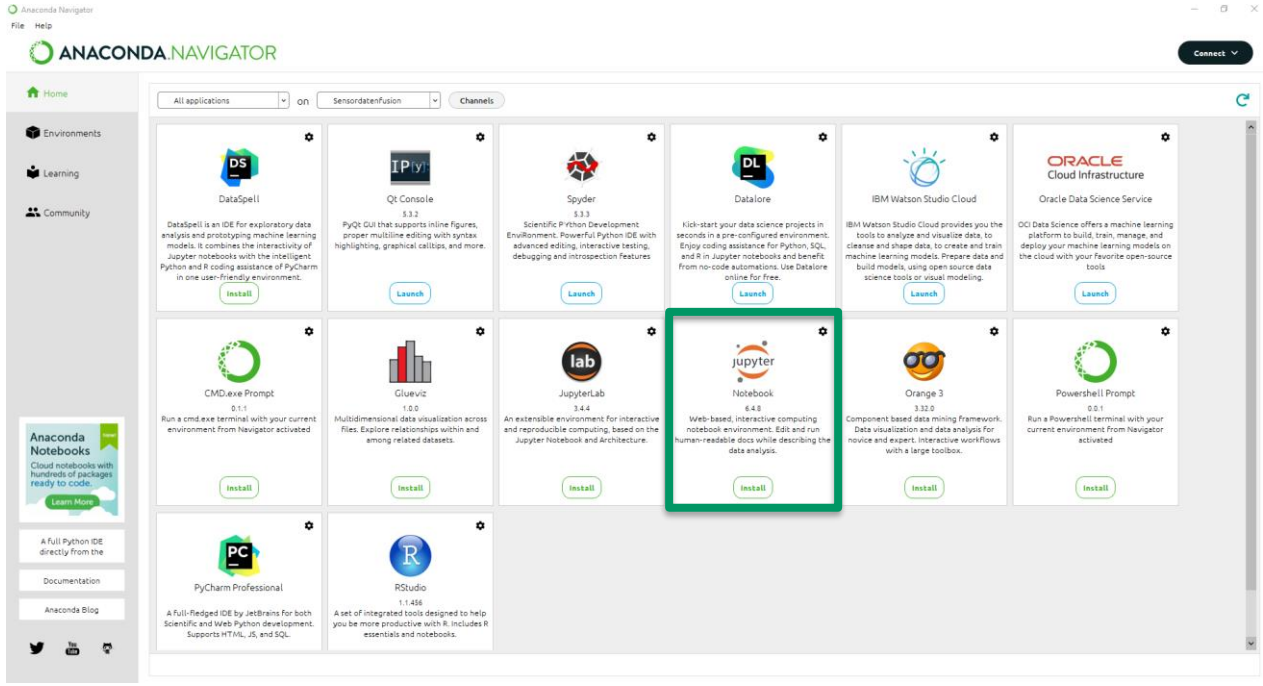
Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

- **You will find two exercises accompanying last week‘s lectures on Moodle.**
- ***Exercise Sheet 1***
  - A couple of very gentle initial exercises (vector/matrix multiplication, Euler angles, derivatives / gradient vectors).
  - Doing this sheet is **highly recommended**, as is handing it in for correction. Depending on number of submissions, I will try to give results on Monday.
- ***ProgExercise 2***
  - A Python program which realizes a Viola Jones face detector.
  - Your task is to use the supplied camera calibration matrix K, and the size of the detected face, to estimate the 3D position of the face in space.
  - This will develop into a longer series of programming exercises estimating positions and velocities of objects in space with Kalman Filters.
  - Doing the exercise therefore is **highly recommended** as well.
- **Solutions for both of the exercises will be posted on Wed Oct 22.**

Software: Python/Anaconda

- For the ML examples we will use today, will use the Jupyter Notebook environment.
- It is recommended you install the Anaconda Python environment and distribution, that installs Jupyter as one of its environments.
- You can find it at https://www.anaconda.com/.



Exercise Sheet
Programming Exercise

Machine Learning: Attention, Region CNNs, YOLO

Measuring Quality

- **Precision and Recall: Classification Quality**
  - ***Precision*** measures *how many classifications are correct*:

$$\text{Precision} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifications}} = \frac{tp}{tp + fp}$$

  - ***Recall*** measures *how many instances were identified:*

$$\text{Recall} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifiable Objects}} = \frac{tp}{tp + fn}$$

  - Example:
    - An image contains 10 cars.
    - 7 were correctly identified.
    - 2 non-car patterns were classified as cars (False Positives).
    - 3 cars were not identified (False Negatives).

Instance Is Detected \ Instance Exists	Yes	No
Yes	True Positive (TP)	False Positive (FP)
No	False Negative (FN)	True Negative (TN)

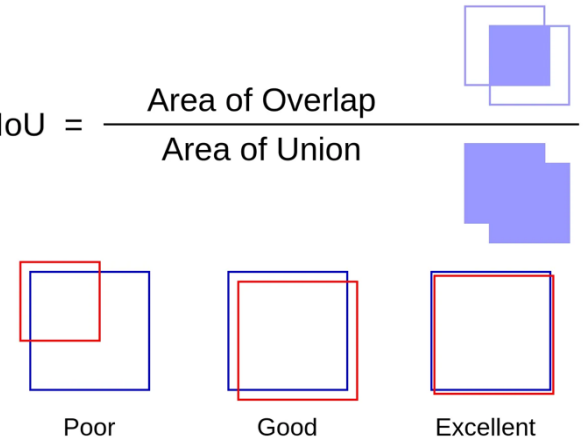
$$\text{Precision} : (7)/(7+2) = 77.8\%$$
$$\text{Recall} : 7/10 = 70\%$$

Measuring Quality

- ***F1 Score*: Classification Quality**
  - The F1 score is the *harmonic mean* of Precision and Recall: $F = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
  - It unifies Precision and Recall into one „goodness score“.

- **Intersection Over Union: Spatial Localization Quality**
  - Measures the normalized overlap of two bounding boxes (e.g. ground truth and prediction).
  - Gives a good estimate on precision of the bounding box extraction, vital for object **spatial localization** .

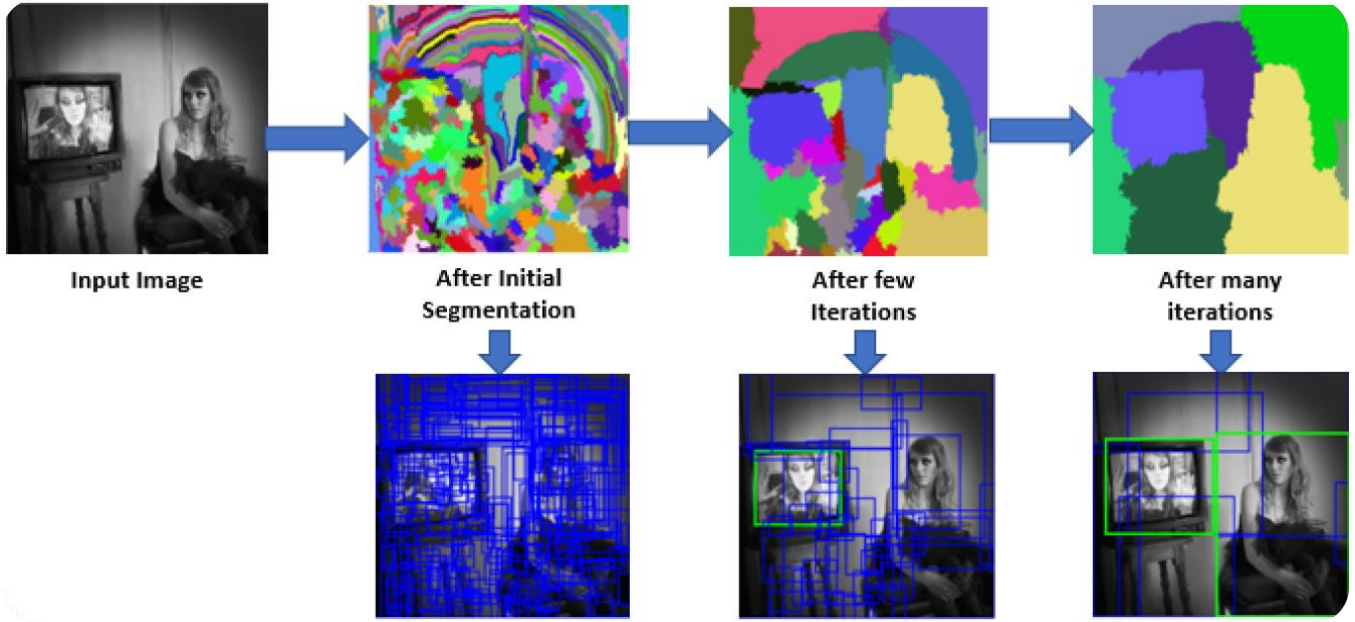
$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



The Trouble with CNNs…

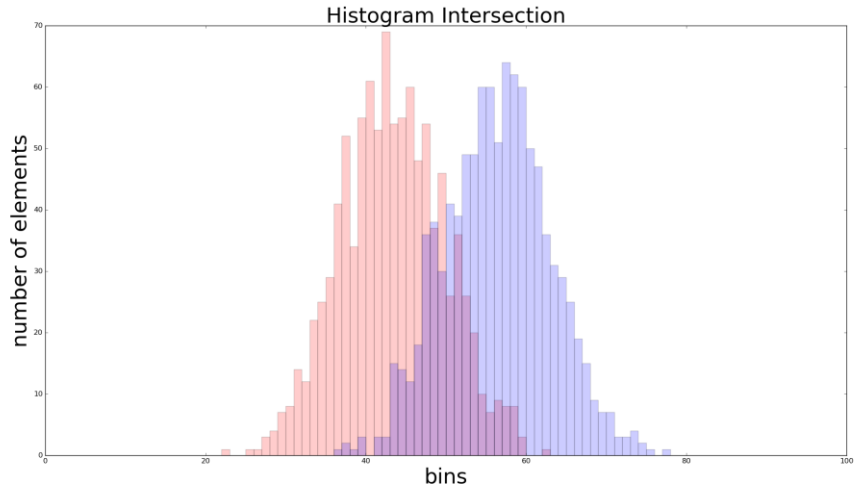
- **CNNs are great to classify objects based on appearance, and they are a lot cheaper than regular Multilayer Perceptrons.**
- **But they are still locked to a certain region given by the number of connections / weights to the fully connected layer.**
- **Pooling layers introduce a bit of scale invariance, but you still need regions of interest.**

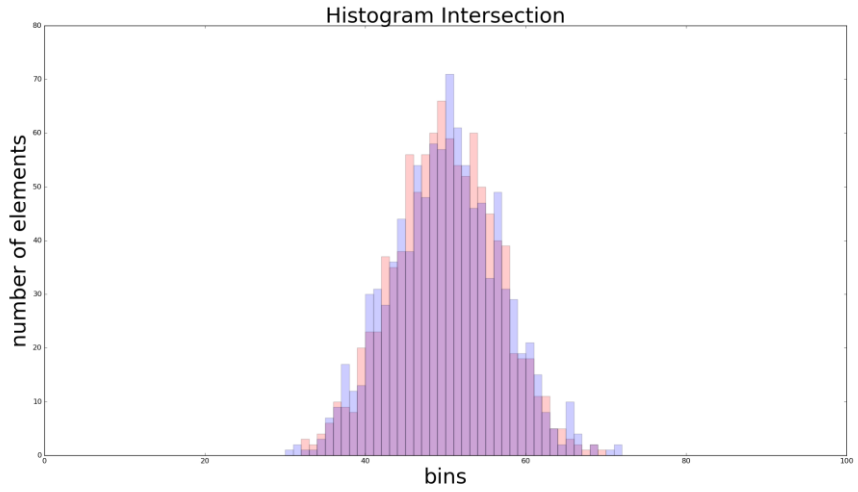
- **We use a *region proposal* algorithm to scan the picture, e.g. Selective Search.**



Uijlings & al.: Selective Search for Object Recognition

- **Color Similarity:**
  - Take the histogram $C_i = \{c_i^1, \ldots, c_i^n\}$ for each region r_i and each of the 3 color channels.
  - Color similarity can be measured by the histogram intersection: $s_{\text{colour}}(r_i, r_j) = \sum_{k=1}^{n} \min(c_i^k, c_j^k)$.





- **Texture Similarity:**
  - Take image derivative in u, v directions and derive gradient direction and strength in each pixel (see lecture 3, Sobel filtering) for each color channel.
  - Build n-channel texture histogram $T_i = t_i^1, \ldots, t_i^n$ of gradient strength over direction for each region i.
  - Compare using histogram intersection: $s_{\text{texture}}(r_i, r_j) = \sum_{k=1}^{n} \min(t_i^k, t_j^k)$
- **Size similarity:**
  - We want small regions to merge early, but not one large region gobbling up all other regions.
  - We define $$s_{\text{size}}(r_i, r_j) = 1 - \frac{\text{size}(r_i) + \text{size}(r_j)}{\text{size}(\text{image})}$$

- **Fill Similarity – how well do the regions fit into each other? Do they largely occupy the same bounding box?**
  - If r_i is contained in r_j, it makes sense to combine to fill holes.
  - If r_i, r_j are hardly touching, they should not be merged.
  - Define BB_{ij} to be the bounding box around r_i and r_j.
  - Fill similarity is then given as $s_{\mathrm{fill}}(r_i, r_j) = 1 - \frac{\mathrm{size}(\mathrm{BB}_{ij}) - \mathrm{size}(r_i) - \mathrm{size}(r_j)}{\mathrm{size}(\mathrm{image})}$.

- **Combine similarities**

$$s(r_i, r_j) = s_{\mathrm{colour}}(r_i, r_j) + s_{\mathrm{texture}}(r_i, r_j) + s_{\mathrm{size}}(r_i, r_j) + s_{\mathrm{fill}}(r_i, r_j)$$

- **Full Algorithm:**

Algorithm 1: Hierarchical Grouping Algorithm

Input: (colour) image
Output: Set of object location hypotheses L

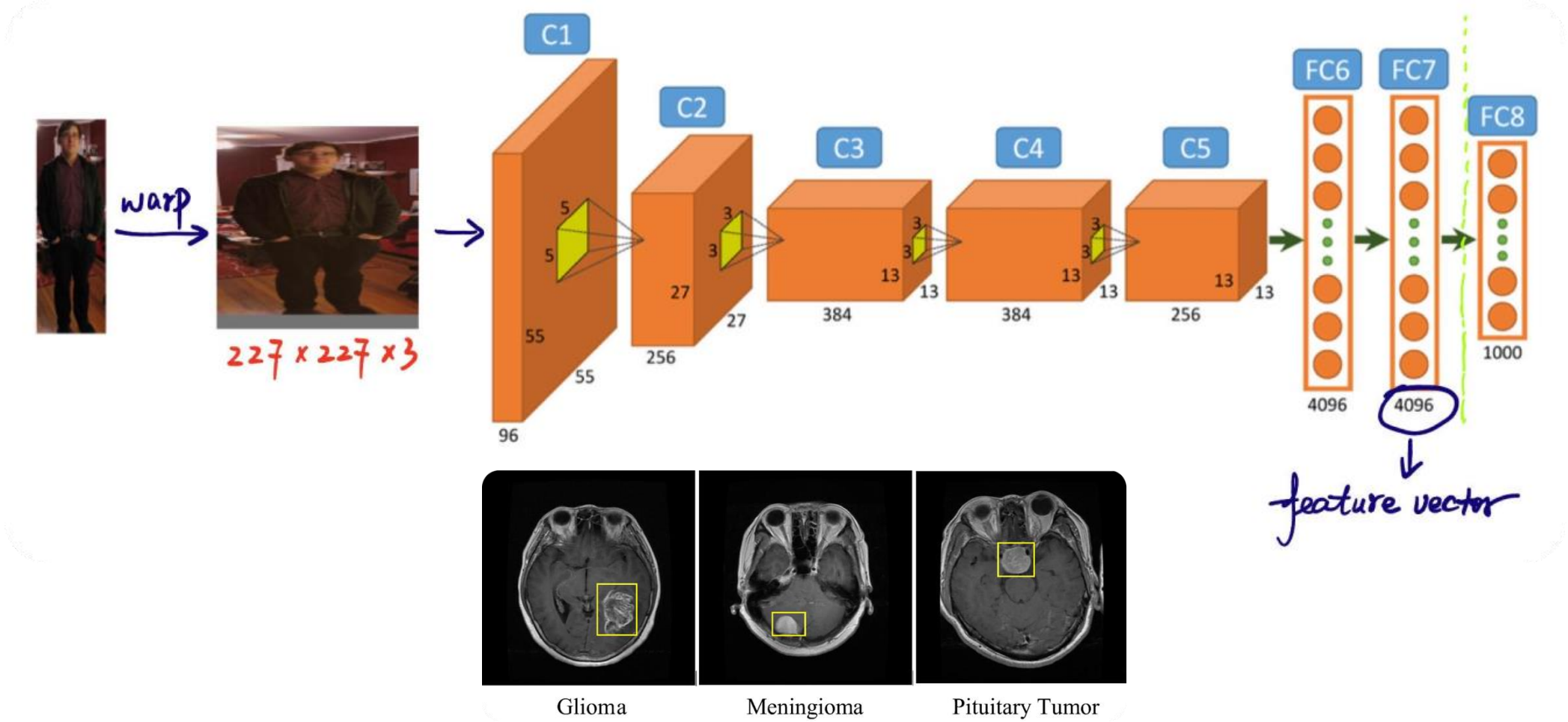
Obtain initial regions $R = \{r_1, \cdots, r_n\}$ using [13]
Initialise similarity set $S = \emptyset$
foreach *Neighbouring region pair* (r_i, r_j) **do**
    Calculate similarity $s(r_i, r_j)$
    $S = S \cup s(r_i, r_j)$

while $S \neq \emptyset$ **do**
    Get highest similarity $s(r_i, r_j) = \max(S)$
    Merge corresponding regions $r_t = r_i \cup r_j$
    Remove similarities regarding $r_i : S = S \setminus s(r_i, r_*)$
    Remove similarities regarding $r_j : S = S \setminus s(r_*, r_j)$
    Calculate similarity set S_t between r_t and its neighbours
    $S = S \cup S_t$
    $R = R \cup r_t$

Extract object location boxes L from all regions in R

[13]: Felzenszwalb, Huttenlocher: Efficient Graph-Based Image Segmentation

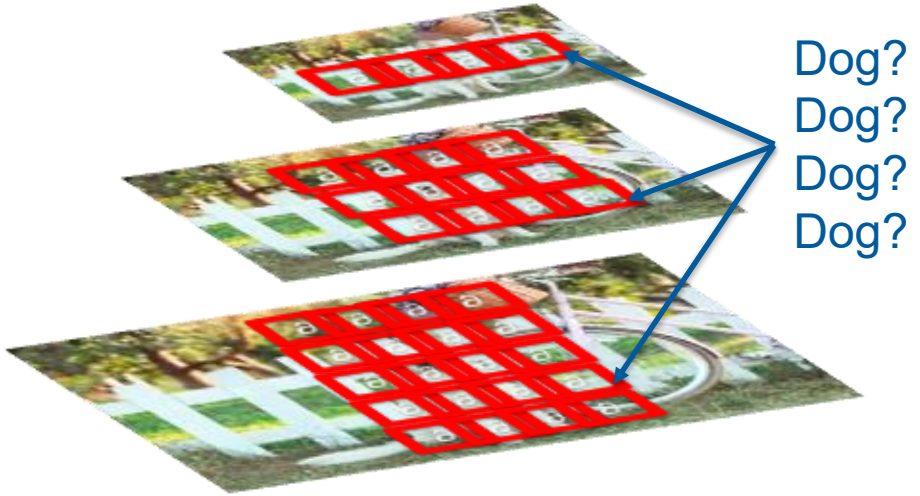
- **Using Regions as input, CNNs can be effectively fed and classify well.**



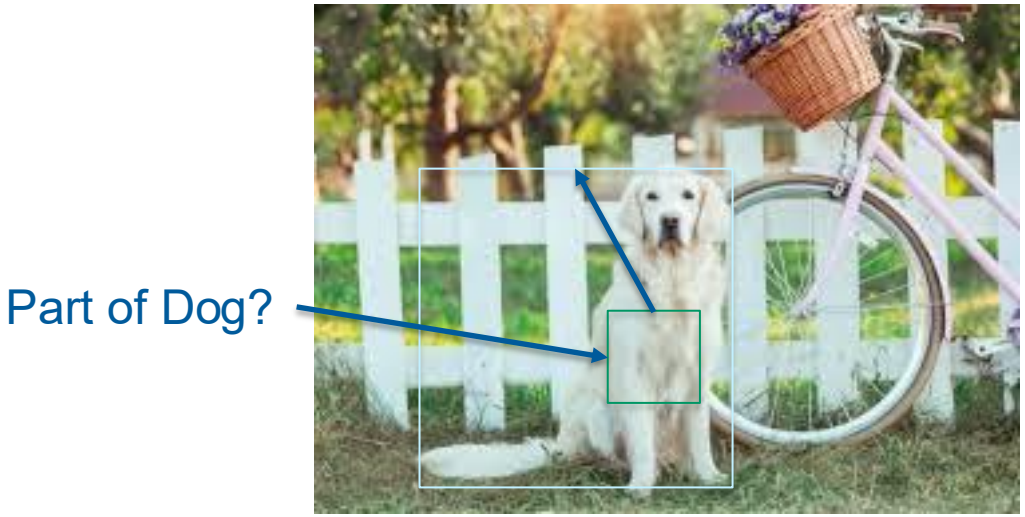
R-CNNs can contribute significantly e.g. to tumor recognition in MRT/CT scans.

Machine Learning: YOLO – A CNN Example

- **R-CNNs use a two-stage approach (region detection → classification).**
- **YOLO (You Only Look Once) and similar networks provide single-shot detection.**
- **Basic idea: If parts of an object can be classified, they can predict bounding boxes for the full object.**



Instead of scanning image pyramids for completely classifiable pictures,

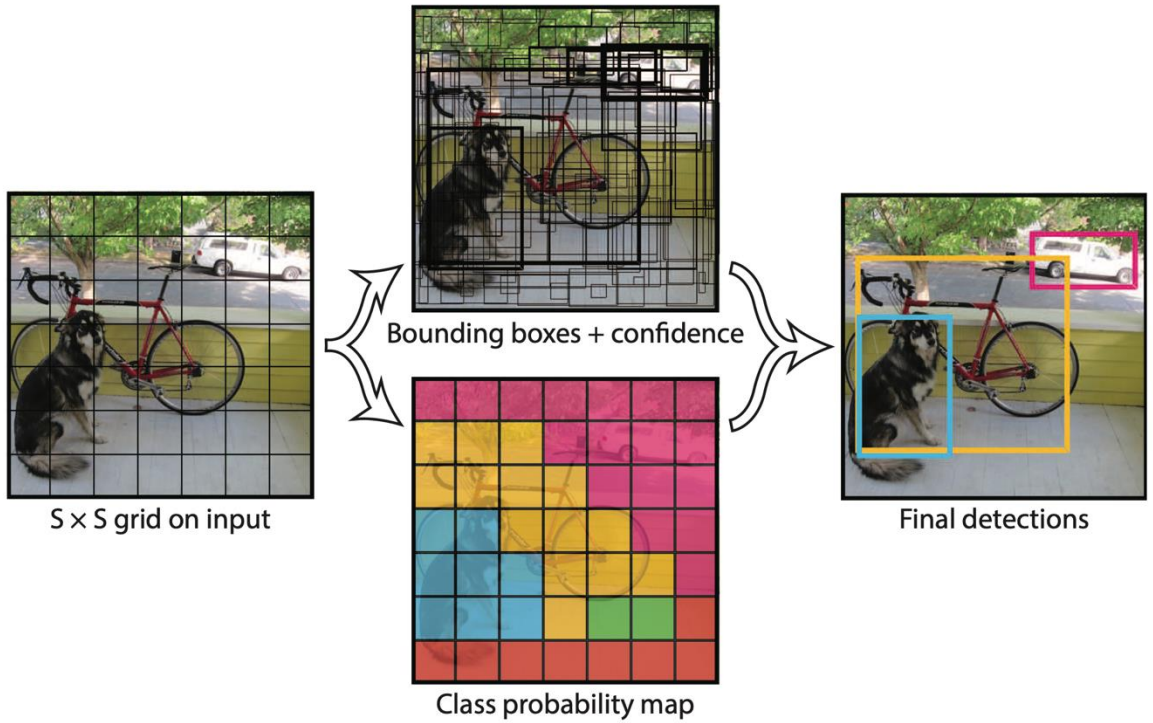


Yolo looks at segments of an image and predicts class and full size bounding box.

- **Yolo divides the input image into an $S \times S$ grid of cells.**
- **Each cell predicts B bounding boxes with 5 properties: $x, y, w, h, \text{confidence}$.**
- **Each cell also predicts C conditional class probabilities, $P(\text{Class}_i|\text{Object})$.**
- **This yields an $S \times S \times (5B + C)$ output.**
- **At test time, these conditional probabilities are multiplied with the box confidence and the IOU vs ground truth:**

$$P(\text{Class}_i|\text{Object}) * P(\text{Object}) * \text{IOU}^{\text{truth}}_{\text{pred}}$$
$$= P(\text{Class}_i) * \text{IOU}^{\text{truth}}_{\text{pred}}$$

…producing a confidence score encoding class probability and bounding box fit.



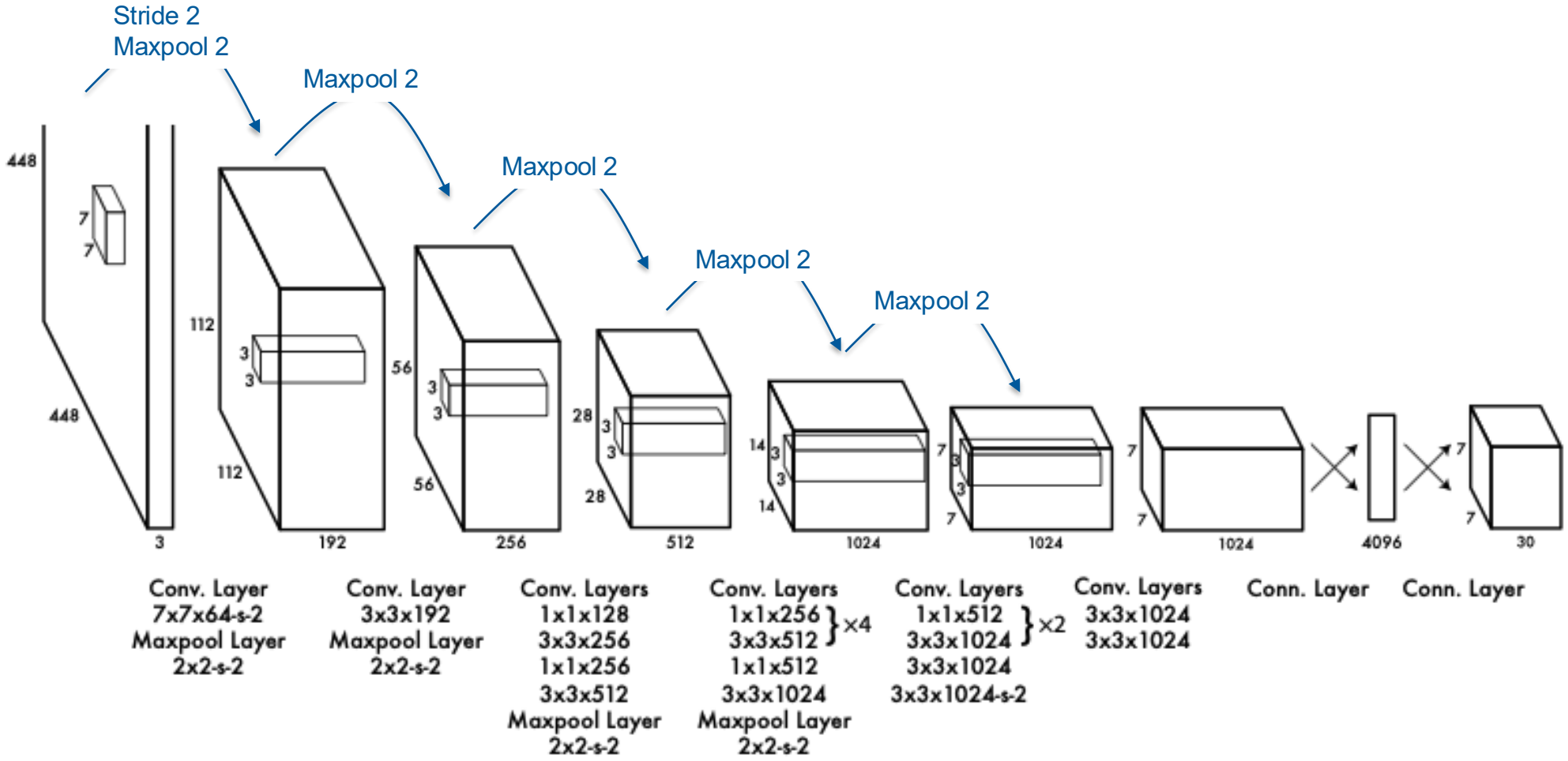
In the original paper, the chosen values were

$S = 7$ grid cells,

$B = 2$ bounding box predictions per cell,

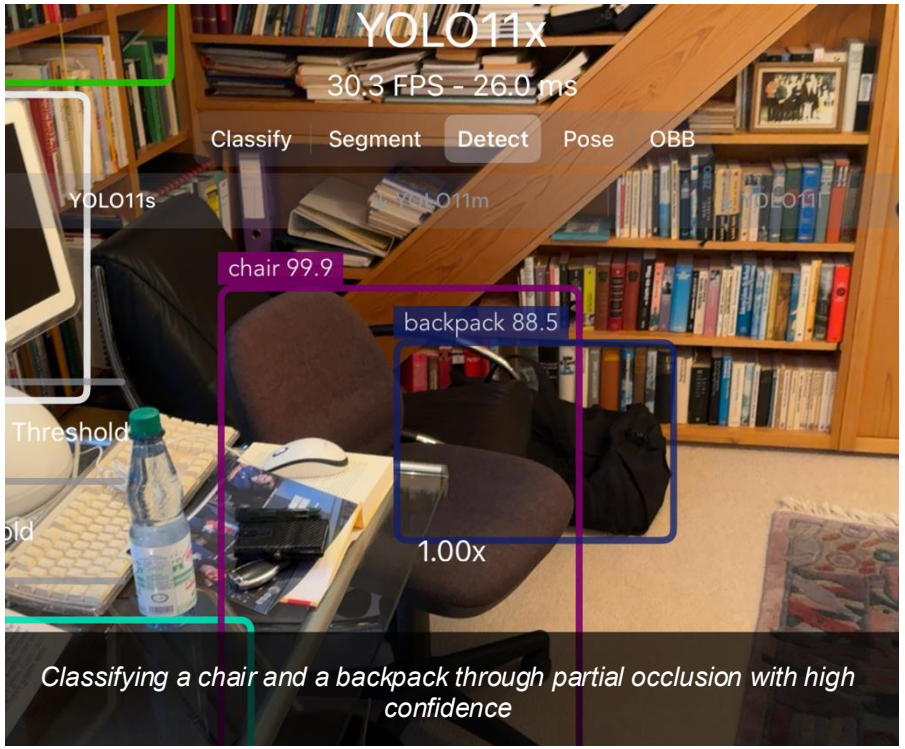
$C = 20$ classes

giving a 7x7x30 output tensor.



YOLO network architecture

- **The segment-to-full approach has the big advantage of being able to classify partially-occluded objects.**



Classifying a chair and a backpack through partial occlusion with high confidence

- **Newer versions of YOLO add**
  - 2D-oriented bounding boxes for a better image region match
  - Keypoint-based skeleton tracking for humans
  - Far improved recognition capacity



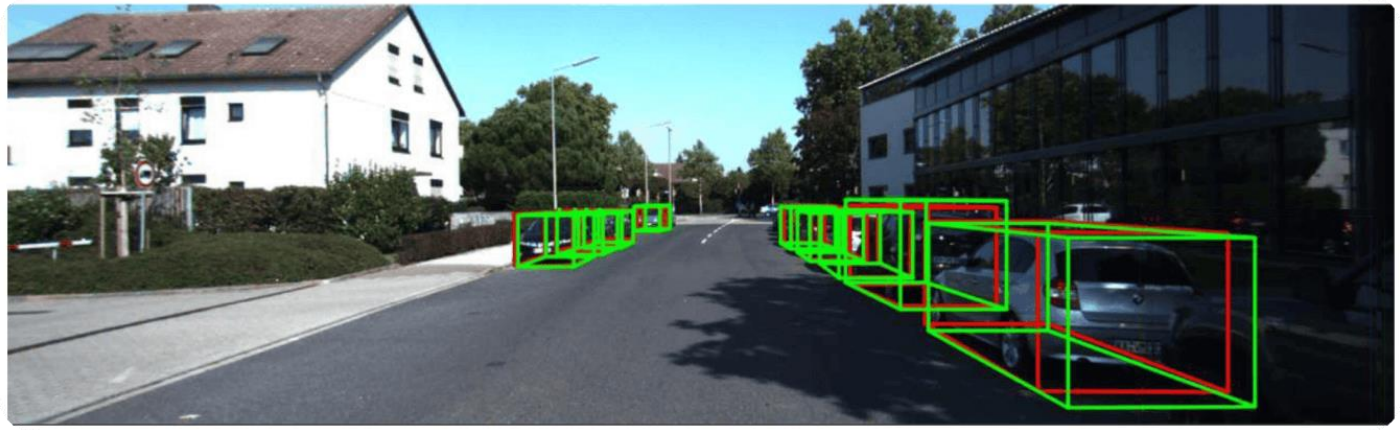


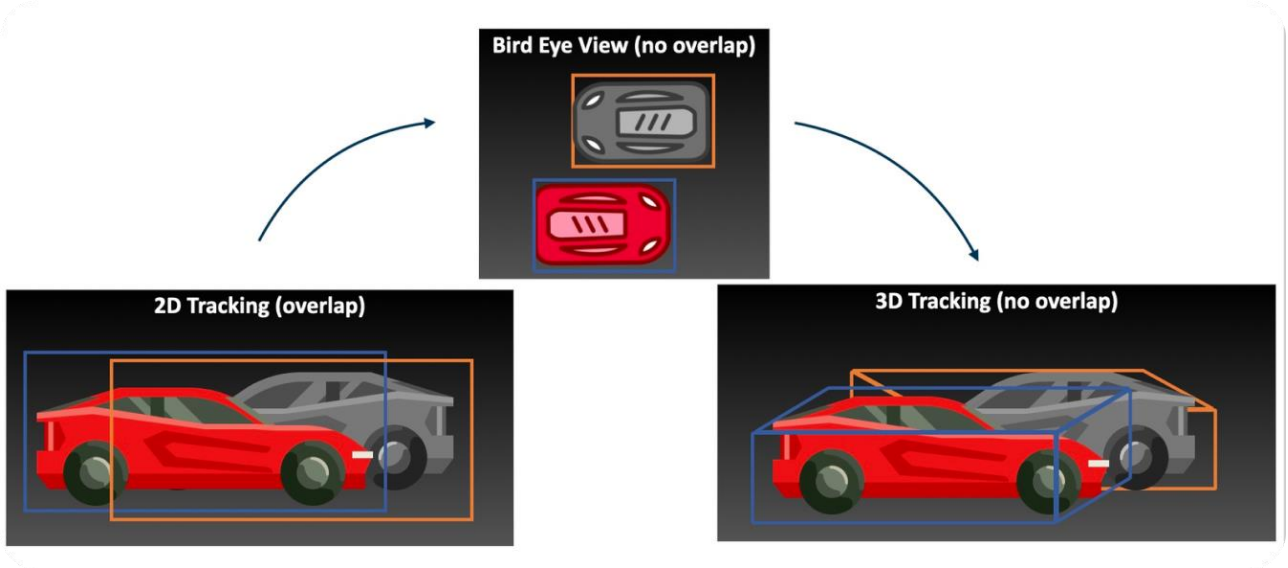
- **YOLO is strong for classifying / detecting many types of objects, and can even be used in real-world robotics or AD scenarios.**
- **This is YOLOv11 running on an example video from the KITTI dataset (https://www.cvlibs.net/datasets/kitti/) that we will look at again.**



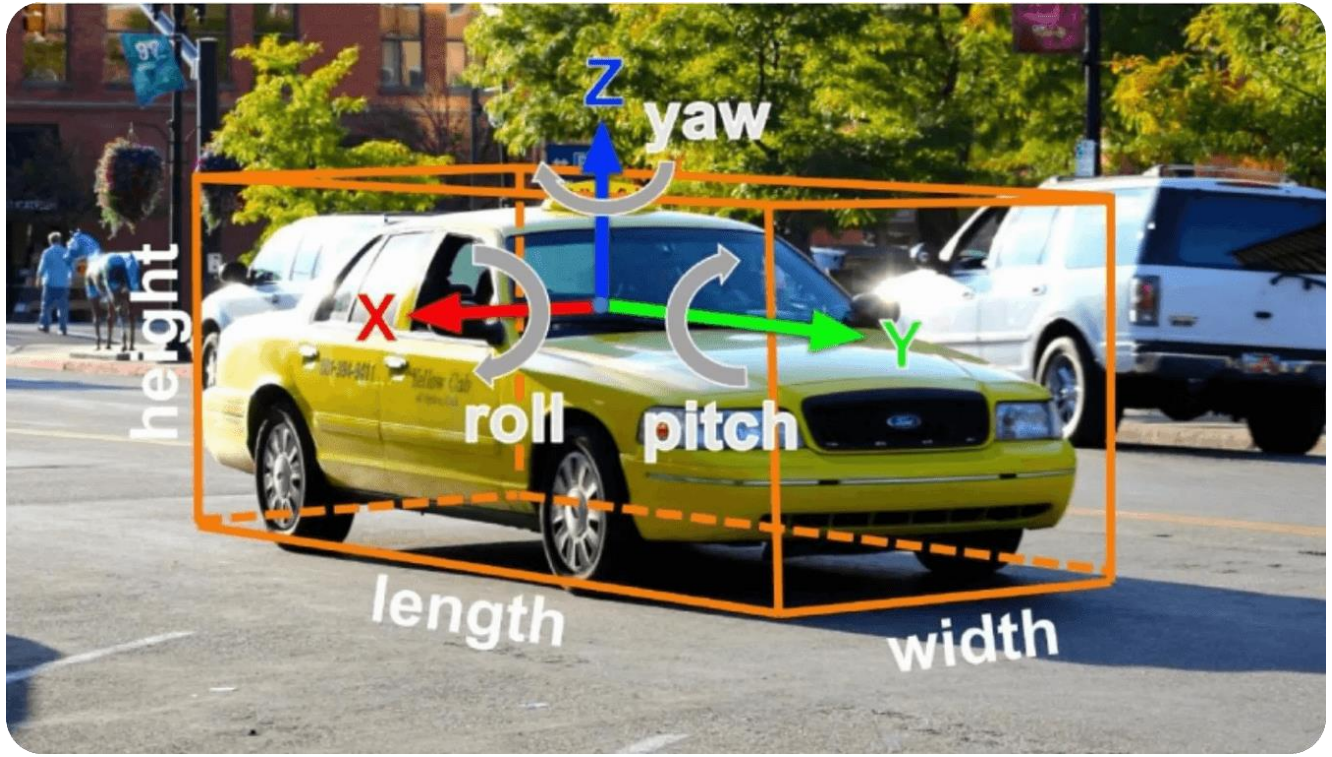
Machine Learning: 3D from 2D

- **In many cases we are interested in 3D information instead of just 2D bounding boxes.**
  - Autonomous Driving
  - Robotics
  - Drones
- **3D information is vital for**
  - Threat / Hazard Assessment
  - Path Planning and Control
  - Approach and Docking
  - Grasping and Manipulation
- **It is also vital for modeling object motion governed by physical laws.**





While YOLO gave us $x, y, w, h, \text{confidence}$, 3D objects need more parameters:



The 3D bounding box is defined by $\underbrace{\mathrm{T} = (t_x, t_y, t_z)^\top}_{\text{Center}}, \underbrace{\mathrm{D} = (d_x, d_y, d_z)^\top}_{\text{Dimension}}, \underbrace{R(\theta, \phi, \alpha)}_{\text{Rotation}}$.

- **We can detect objects using R-CNN or YOLO, giving us the 2D bounding box.**
- **The vertices of the 3D bounding box can be described by**

$$\begin{aligned}\mathbf{X_1} &= (d_x/2, d_y/2, d_z/2)^\top, \\ \mathbf{X_2} &= (-d_x/2, d_y/2, d_z/2)^\top, \\ &\dots, \\ \mathbf{X_8} &= (-d_x/2, -d_y/2, -d_z/2)^\top\end{aligned}$$

and the projections are given by

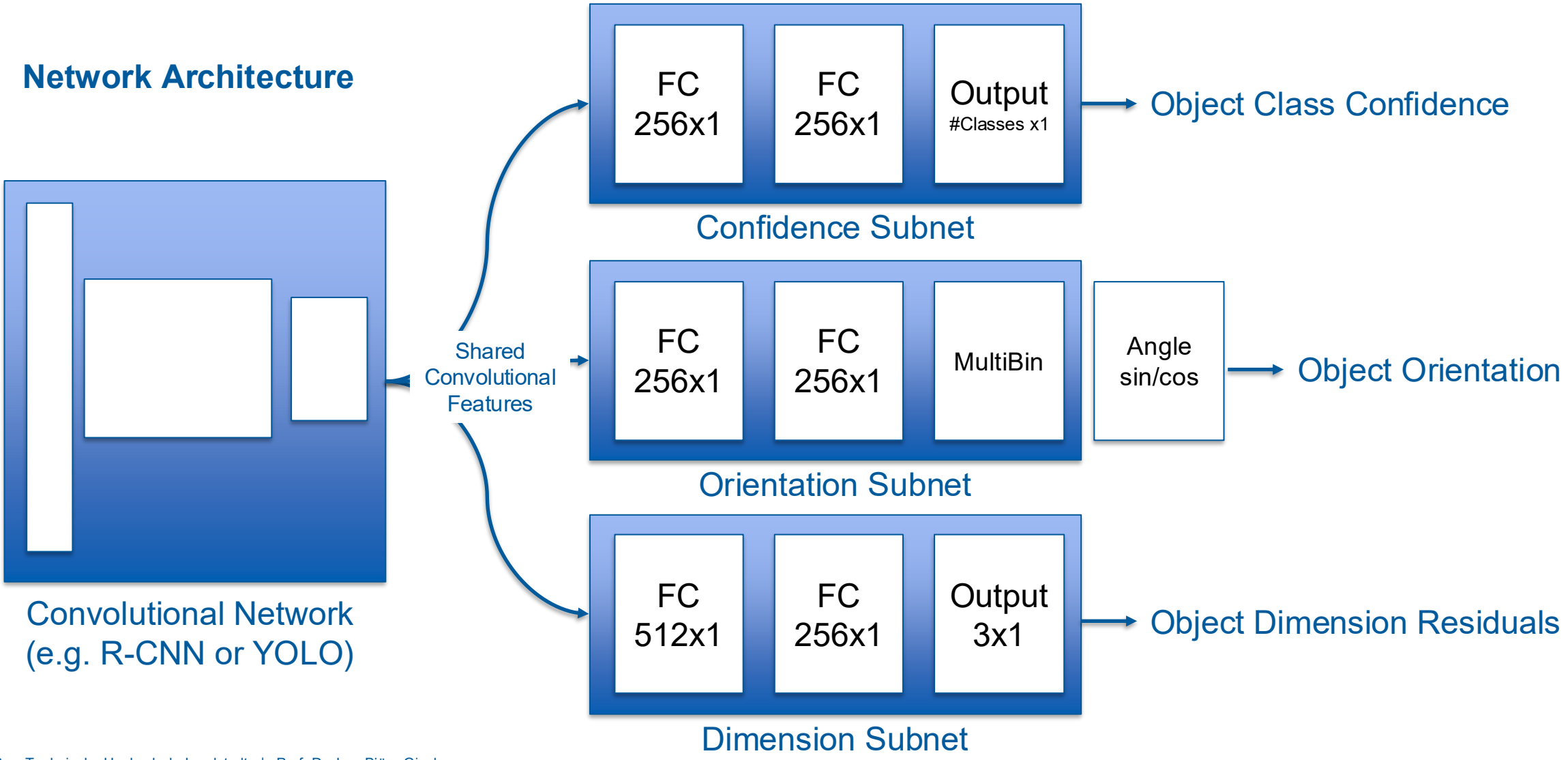
$$\begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix} = \left(K[Rt] \begin{pmatrix} x_i \\ y_i \\ z_i \\ 1 \end{pmatrix} \right)$$



- **Unfortunately, we do not know which 3D corners coincide with with corners of the 2D bounding box.**
- **There are $8^4 = 4096$ possible combinations.**
- **Even if we did, this would give us 4 constraints for 9 DOF.**
- **We cannot solve directly, however we can learn.**

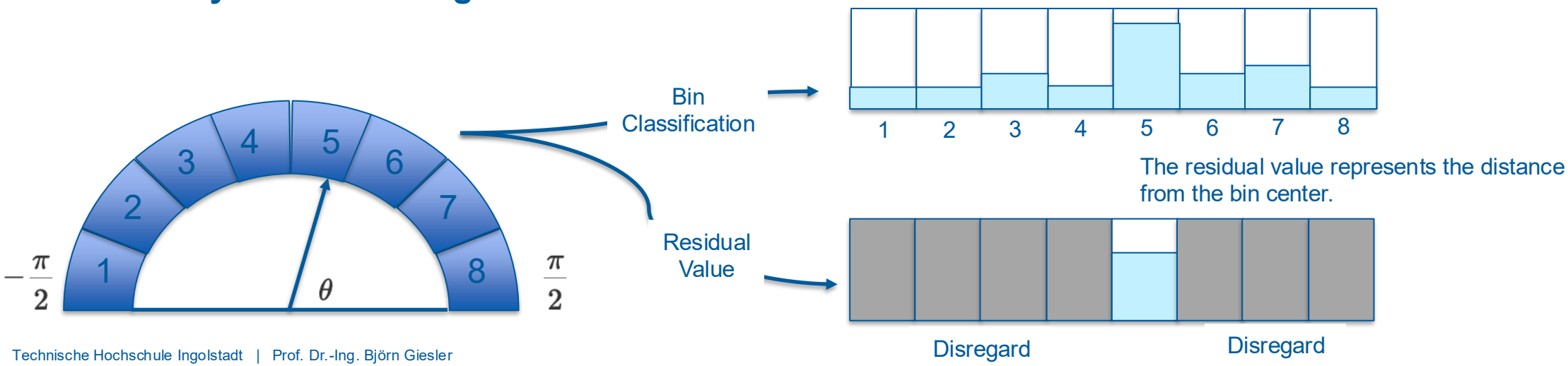


Network Architecture



Multi-Bin Orientation Estimation

- **A Multi-Bin NN is a type of regression architecture commonly used for orientation estimation.**
- **It divides a continuous variable — typically an angle or pose — into several discrete bins, and then predicts both:**
  - Which bin the true value belongs to (classification), and
  - How far inside that bin the value lies (regression).
- **This approach helps the network handle periodic or continuous quantities (like angles) more effectively than direct regression.**



Multi-Bin Orientation Estimation

- **How to encode the residual angle $\delta_i = \theta_{\text{obj}} - \theta_{\text{bin}}$?**
- **The easiest way is to simply encode the angle directly.**
- **This leads to unstable predictions due to direct influence of network noise and uncertainty.**
- **A more robust solution is to**
  - encode $\sin(\delta_i), \cos(\delta_i)$ instead, receiving $\hat{s}_j, \hat{c}_j$ as the prediction for detected object O_j ,
  - enforce unity of the resulting 2-vector $$\sqrt{\sin(\delta_i)^2 + \cos(\delta_i)^2} = 1 \Longrightarrow \begin{bmatrix} \hat{s}'_j \\ \hat{c}'_j \end{bmatrix} = \frac{1}{\sqrt{\hat{s}_j^2 + \hat{c}_j^2}} \begin{bmatrix} \hat{s}_j \\ \hat{c}_j \end{bmatrix}$$
  - and finally obtaining the angle from $\hat{\theta}_{\text{obj},j} = \hat{\theta}_{\text{bin},j} + \sin^{-1} \hat{s}'_j$.

Multi-Bin Orientation Estimation

- **For N bins, the output for the network is of size 3xN:** $[p_1, \ldots, p_N, s_1, \ldots, s_N, c_1, \ldots, c_N]$ **where** p_i **are the bin probabilities and** s_j, c_j **are sines and cosines of the residuals.**
- **The error function is represented as a combination:** $E = E_{\text{bin}} + \lambda \cdot E_{\text{residual}}$ **, where**
  - E_{bin} is the binning error and
  - $E_{\text{residual}} = \cos(\theta_{truth} - \hat{\theta}_{bin} - \hat{\delta})$ is the error obtained from the angle calculated from predicted bin and residual.
  - A stabilizing extension of the algorithm is *overlapping the bins*, representing each angle with multiple (e.g. 2) bin / residual combinations, and interpolating between the two.

Method	Pros	Cons
Direct angle regression	Simpler Outputs continuous values directly	Poor handling of periodicity Struggles with multimodal distributions
Multibin network	Handles periodicity (0°=360°) More stable learning Better accuracy	Slightly more complex Bin choice matters

Learning Dimension

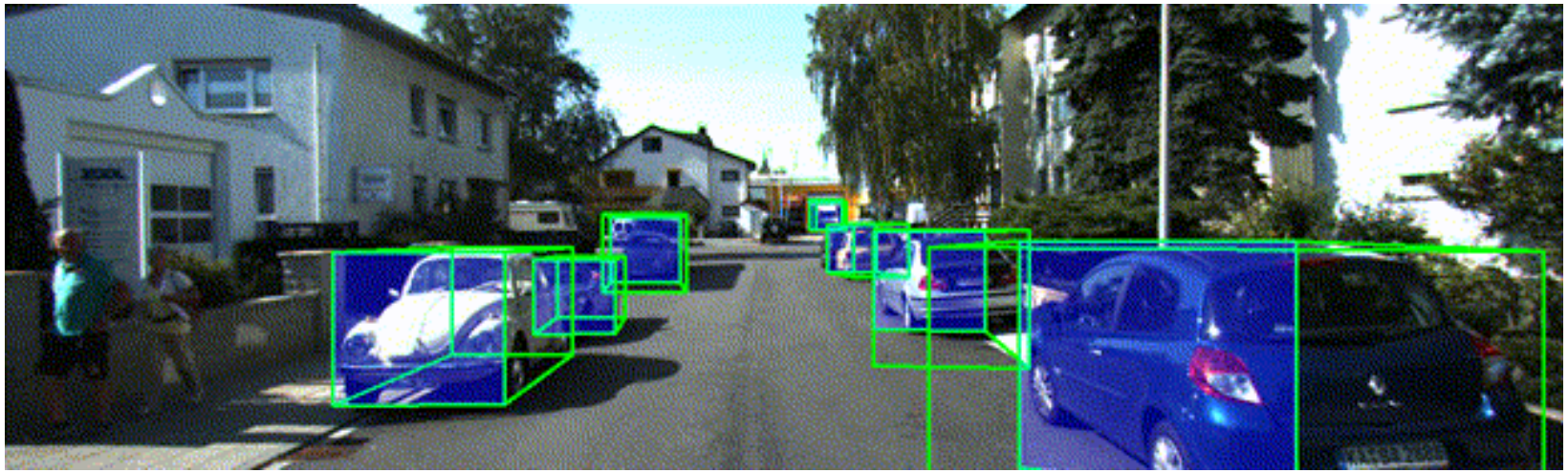
- **Dimension is tied strongly to object class. Cars, trucks, bicyclists are a certain size, even humans.**

- **We therefore only learn the residual $\mathbf{\Delta}_i$ to the mean dimensions $\bar{\mathbf{D}}_\mathrm{c} = \frac{1}{N_\mathrm{c}} \sum \begin{bmatrix} d_{x,\mathrm{c},i} \\ d_{y,\mathrm{c},i} \\ d_{z,\mathrm{c},i} \end{bmatrix}$ for object class c:**

$$\mathbf{\Delta}_i = \mathbf{D}_i - \bar{\mathbf{D}}_\mathrm{c}$$

with the Error or Loss function $E_\mathrm{dims} = \frac{1}{n} \sum (\mathbf{D}_{\mathrm{truth},i} - \bar{\mathbf{D}}_\mathrm{c} - \hat{\mathbf{\Delta}}_i)$.

- **Using the obtained size and rotation, the full 3D information can be obtained with the following logic:**
  - There are 4096 total combinations of corner matches.
  - Assuming the object is always upright,
    - Top and bottom corners of the bounding box can only correspond to top and bottom corners of the object, reducing the number of options to 1024.
    - Left and right corners of the bounding box can only correspond to left and right corners of the object, reducing to 256.



Results (slowed down for visualization)
From sample implementation in [2]



Features that contribute most to orientation estimation

Appendix
Sources

1. **A. Mosavian, D. Anguelov, J. Flynn: 3D Bounding Box Estimation Using Deep Learning and Geometry, https://arxiv.org/pdf/1612.00496**
2. **https://github.com/ruhyadi/YOLO3D**



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!